An Artificial Neural Network Representation of the SABR Stochastic Volatility Model

Replication and Extensions

Seminar in Applied Quantitative Finance

Cecilia Trojani, Auguste Darracq Paries and Petrit Bllaca

November 2025

Abstract

This report presents a replication and some methodological extensions of *"An Artificial Neural Network Representation of the SABR Stochastic Volatility Model"* by [McGhee, 2020]. In the original study, neural networks were first trained on implied volatilities computed via the Hagan asymptotic approximation and then refined using data generated by a two-factor finite-difference (2FDM) solver for the SABR dynamics. Our work reproduces these main results and broadens both the empirical and numerical scope of the analysis.

Our contributions are fourfold. First, we reimplement the training pipeline for neural networks calibrated on the Hagan approximation, strictly following the architecture and methodology of [McGhee, 2020]. The analysis is carried out both for the classical lognormal case $\beta = 1$ and for the full elasticity range $\beta \in [0, 1]$, enabling a systematic assessment of how approximation accuracy depends on the variance elasticity. Second, we investigate the effect of model capacity by training networks with varying hidden-layer widths, thereby identifying the regime in which approximation quality deteriorates due to insufficient representational power and the point beyond which additional width yields diminishing returns. Third, we construct numerically generated high-fidelity training datasets using a two-factor finite-difference solver for the SABR model and train neural networks directly on these numerically obtained implied volatilities, again considering both $\beta = 1$ and $\beta \in [0, 1]$. Finally, we extend the original framework by incorporating a conditional-integration method for computing SABR implied volatilities. Although this approach is computationally efficient, the resulting dataset is less globally smooth, and neural networks trained on it achieve noticeably lower refinement accuracy than those trained on 2FDM data.

Overall, this study complements and expands the results of [McGhee, 2020] by providing a broader empirical evaluation of neural-network–based volatility approximations across elasticity parameters, network capacities, and reference pricing models, while maintaining a focus on computational efficiency and robustness.

1 Introduction

The stochastic alpha–beta–rho (SABR) model of [Hagan et al., 2002] is a foundational stochastic-volatility framework for modelling the joint evolution of a forward price and its instantaneous volatility. Its analytical implied-volatility approximation, also introduced by [Hagan et al., 2002], offers a fast closed-form expression that has become a standard tool for calibration and risk management. Despite its efficiency, the Hagan formula loses accuracy for long maturities, extreme moneyness, and high vol-of-vol, where it can generate non-physical smiles. These limitations motivate the search for surrogate pricing techniques that retain closed-form speed while matching PDE-level accuracy.

[McGhee, 2020] proposed a hybrid data-driven approach based on feed-forward neural networks to address these shortcomings. In this framework, the network is first pre-trained on implied volatilities generated by the Hagan approximation and subsequently refined using synthetic data produced by a two-factor finite-difference (2FDM) solver for the SABR dynamics. The resulting two-stage surrogate combines the speed of the analytical formula with the accuracy of a PDE-based benchmark. In this report, we reproduce and extend the methodology of [McGhee, 2020] with four main objectives:

1. **Hagan-based neural networks for $\beta = 1$ and $\beta \in [0, 1]$.** We reimplement the ANN surrogate trained on Hagan implied volatilities for the classical lognormal case $\beta = 1$ and extend the training pipeline to the full elasticity range $\beta \in [0, 1]$. This enables a systematic investigation of how approximation accuracy evolves as the SABR dynamics interpolate between the lognormal, CEV, and normal regimes.

2. **Architectural ablation across hidden-layer widths.** We analyse the effect of representational capacity by training networks with a wide range of hidden-layer widths, identifying the thresholds at which underfitting occurs and the point beyond which performance gains become marginal.

3. **2FDM-based neural networks for $\beta = 1$ and $\beta \in [0, 1]$.** We generate high-fidelity implied-volatility datasets using the two-factor finite-difference solver of [McGhee, 2020] and train ANNs directly on these numerical prices. The analysis is carried out for the full interval $\beta \in [0, 1]$, enabling a direct comparison between analytical and numerical training targets.

4. **Conditional-integration surrogate.** Because constructing 2FDM datasets is computationally demanding, we incorporate the semi-analytical conditional-integration method of [McGhee, 2011]. This provides an additional high-accuracy and computationally lighter reference model for implied-volatility generation, complementing both the Hagan approximation and the 2FDM solver.

All datasets, numerical experiments, and trained models used in this study are publicly available in our GitHub repository:

`https://github.com/AugusteDP-git/SABR_ANN`

ensuring full reproducibility and providing a reference implementation for future work.

Taken together, these methodological extensions allow for a comprehensive assessment of (i) the robustness of neural-network surrogates across elasticity parameters, (ii) the interplay between approximation accuracy and network capacity, and (iii) the relative merits of analytical, numerical, and integration-based pricing mechanisms for constructing efficient and reliable SABR implied-volatility surfaces.

2 Theoretical Background

This section summarizes the theoretical foundations underlying the numerical and empirical analysis of this report. We recall (i) the stochastic-volatility dynamics of the SABR model, (ii) the analytical implied-volatility approximation of [Hagan et al., 2002], (iii) the universal-approximation properties of feed-forward neural networks motivating their use as smooth surrogate models, (iv) the conditional-integration method of [McGhee, 2011], and (v) the two-factor finite-difference (2FDM) scheme used as a high-fidelity numerical benchmark.

2.1 SABR stochastic-volatility model

The SABR model describes the joint evolution of the forward price F_t and its volatility σ_t under the following dynamics:

$$\begin{aligned} dF_t &= \sigma_t F_t^{\beta}\, dW_t \ ; \\ d\sigma_t &= \xi\, \sigma_t\, dZ_t \ ; \\ d\langle W, Z\rangle_t &= \rho\, dt \ ; \end{aligned}$$

where $\beta \in [0,1]$ is the elasticity parameter, $\xi > 0$ the volatility of volatility, and $\rho \in [-1,1]$ the instantaneous correlation between the Brownian drivers W_t and Z_t. The parameter β controls the local elasticity of volatility with respect to the forward, interpolating between the *normal* case ($\beta = 0$) and the *lognormal* case ($\beta = 1$). The flexibility of the SABR specification enables a rich range of smile behaviours: high skew for small β, asymptotic lognormal tails for $\beta \approx 1$, and strong curvature for large ξ or short maturities.

2.2 Hagan implied-volatility approximation

[Hagan et al., 2002] derived an asymptotic closed-form approximation for SABR implied volatilities. The expansion is exact to first order in maturity and provides a fast, analytically tractable mapping

$$(K, F_0, T, \alpha, \beta, \rho, \xi) \ \longmapsto \ \sigma_{\text{imp}}(K, F_0, T)\ .$$

In the general elasticity case $\beta \in [0,1]$, the implied volatility can be written as

$$\sigma_{\text{imp}}(K,F_0,T) = \frac{\alpha z}{q(K,F_0)\, x(z)} \left[1 + \left(\frac{(1-\beta)^2}{24} \frac{\alpha^2}{(F_0K)^{1-\beta}} + \frac{\rho\beta\xi\alpha}{4(F_0K)^{(1-\beta)/2}} + \frac{2-3\rho^2}{24}\xi^2 \right) T + \dots \right],$$

where

$$z = \frac{\xi}{\alpha} \frac{F_0^{1-\beta} - K^{1-\beta}}{1-\beta}, \qquad x(z) = \log\left(\frac{\sqrt{1-2\rho z + z^2} + z - \rho}{1-\rho} \right),$$

and

$$q(K,F_0) = (F_0K)^{\frac{1-\beta}{2}} \left[1 + \frac{(1-\beta)^2}{24} \log^2\left(\frac{F_0}{K}\right) + \frac{(1-\beta)^4}{1920} \log^4\left(\frac{F_0}{K}\right) + \dots \right].$$

Lognormal case ($\beta = 1$). When $\beta \to 1$, the following holds:

$$(F_0K)^{(1-\beta)/2} \to 1, \qquad \frac{F_0^{1-\beta} - K^{1-\beta}}{1-\beta} \to \log(F_0/K),$$

yielding the classical lognormal SABR formula

$$\sigma_{\text{imp}}(K,F_0,T) = \alpha\, \frac{z}{x(z)} \left[1 + \left(\frac{\rho\xi\alpha}{4} + \frac{2-3\rho^2}{24}\xi^2 \right) T + \dots \right] .$$

Limitations. Despite its analytical convenience, the Hagan approximation has well-known failure modes: large vol-of-vol ξ, long maturities, extreme moneyness levels, and strong correlation can lead to distorted smile shapes or even arbitrage violations (e.g., negative densities). For these regions, high-fidelity numerical benchmarks—such as finite-difference PDE solvers or semi-analytic integration schemes—are preferable.

2.3 Neural-network universal approximation principle

The use of feed-forward neural networks as surrogate models for the SABR implied-volatility mapping is supported by the classical *universal approximation theorem*. In its standard form, the theorem states that a single-hidden-layer network with any continuous, non-polynomial activation function (such as Softplus, tanh, or ReLU) can approximate every continuous function on a compact domain to arbitrary accuracy. Formally,

$$\forall\, \varepsilon > 0 \quad \exists\, W, b \quad \text{s.t.} \quad \|f(x) - \mathrm{NN}(x; W, b)\|_\infty < \varepsilon\,.$$

In our context, the Hagan implied-volatility formula defines a smooth mapping

$$(T,\, \sigma_0,\, \xi,\, \rho,\, x_1, \ldots, x_{10}) \;\longmapsto\; \sigma_{\mathrm{imp}}\ ,$$

and the training domain is compact by construction, since all inputs are sampled over bounded parameter intervals. Consequently, a sufficiently wide feed-forward network is theoretically capable of replicating the full implied-volatility surface with arbitrarily low error, while preserving the smoothness inherent in the analytical approximation. This theoretical guarantee provides the motivation for the width-ablation study in Section 4, where we investigate empirically how network capacity influences approximation accuracy, stability, and error behaviour across different maturities and moneyness levels.

2.4 Integration-based semi-analytical method

An accurate alternative to finite-difference schemes for computing SABR implied volatilities is the conditional-integration approach of [McGhee, 2011]. The method exploits the tractable structure of the lognormal volatility process to represent the price as a nested integration over the terminal volatility σ_T and the mean integrated variance. The method exploits the fact that the volatility process is lognormal for all elasticity parameters β,

$$d\sigma_t = \xi \sigma_t\, dW_t^\sigma\ ,$$

while the forward dynamics become tractable in the lognormal SABR case ($\beta = 1$), where

$$dF_t = \sigma_t F_t\, dW_t\ .$$

In this setting, the option price can be expressed as a nested integration over the terminal volatility σ_T and the mean integrated variance. Since the volatility process is lognormal, we have the exact distribution

$$\log \sigma_T \sim \mathcal{N}\big(\log \sigma_0 - \tfrac{1}{2}\xi^2 T,\ \xi^2 T\big)\ .$$

Consequently, the outer integration with respect to σ_T may be efficiently approximated using a Gaussian–Hermite quadrature. To obtain the conditional representation of the forward, we start from

$$d \ln F(t,T) = -\tfrac{1}{2}\sigma(t)^2\, dt + \sigma(t)\, dW(t)\ ,$$

and decompose the Brownian driver as follows:

$$dW(t) = \rho\, dW_\sigma(t) + \sqrt{1-\rho^2}\, dW_F(t) \equiv \rho\, dW_\sigma(t) + \bar{\rho}\, dW_F(t)\ .$$

Integration yields:

$$F(T,T) \mid \sigma(0,T) = F(0,T)\exp\!\left(-\tfrac{1}{2}\vartheta(T)T + \rho\int_0^T \sigma(t)\,dW_\sigma(t)\right)\exp\!\left(\bar{\rho}\int_0^T \sigma(t)\,dW_F(t)\right),$$

where

$$\vartheta(T) = \frac{1}{T}\int_0^T \sigma(t)^2\,dt$$

is the mean integrated variance. Moreover, for the lognormal volatility driven by

$$d\sigma(t) = \xi\,\sigma(t)\,dW_\sigma(t)\ ,$$

the stochastic term simplifies exactly to:

$$I_{\sigma W} = \int_0^T \sigma(t)\,dW_\sigma(t) = \int_0^T \frac{d\sigma(t)}{\xi} = \frac{1}{\xi}\big(\sigma(T)-\sigma(0)\big)\ .$$

Thus, the option price can be expressed as an integration over the joint density of σ_T and $\vartheta(T)$. To evaluate the inner conditional expectation, McGhee proposes a lognormal approximation calibrated by matching its first two conditional moments. Let $\sigma(t) \mid \sigma_T$ follow the Brownian–bridge representation

$$\sigma(t) \mid \sigma(T) = \sigma(0)\,\exp\!\left(-\tfrac{1}{2}\xi^2 t + \frac{\xi t}{\sqrt{T}}\,\zeta + \xi\Big(1-\tfrac{t}{T}\Big)^{1/2} W_\sigma(t)\right),$$

where

$$\zeta = \frac{\ln\sigma(T) - \ln\sigma(0) + \tfrac{1}{2}\xi^2 T}{\xi\sqrt{T}}\ .$$

The first conditional moment is

$$\mathbb{E}\left[\frac{1}{T}\int_0^T \sigma(t)^2\,dt \;\middle|\; \sigma(T)\right] = \frac{\sigma^2(0)}{T}\frac{\sqrt{2\pi T}}{2\xi}\exp\!\left(\tfrac{1}{2}q^2\frac{\xi^2}{T}\right)\big[N(\phi(T)) - N(\phi(0))\big]\ ,$$

where $N(\cdot)$ is the standard normal CDF and

$$\phi(t) = \frac{2\xi}{\sqrt{T}}\left(t - \tfrac{1}{2}q\right), \qquad q = T + \frac{1}{\xi^2}\log\!\left(\frac{\sigma(T)}{\sigma(0)}\right).$$

The exact second moment ([Lyu, 2020]) is given by the following closed-form expression:

$$\begin{aligned}
\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma(t)^2\,dt\right)^2 \;\middle|\; \sigma(T)\right] = \frac{1}{T^2}\,\sigma^4(0)\,e^{c_0}\frac{\sqrt{2\pi T}}{\xi}\Bigg\{ & \frac{1}{c_1}e^{c_1 T}\big[N(p_0+p_1T) - N(p_2+p_3T)\big] \\
& - \frac{1}{c_1}\big[N(p_0) - N(p_2)\big] \\
& - \frac{1}{c_1}\exp\!\big[-\tfrac{1}{2}\left(p_0^2 - (p_0p_1-c_1)^2/p_1^2\right)\big]\left[N\!\left(p_1T + \frac{p_0p_1-c_1}{p_1}\right) - N\!\left(\frac{p_0p_1-c_1}{p_1}\right)\right] \\
& + \frac{1}{c_1}\exp\!\big[-\tfrac{1}{2}\left(p_2^2 - (p_2p_3-c_1)^2/p_3^2\right)\big]\left[N\!\left(p_3T + \frac{p_2p_3-c_1}{p_3}\right) - N\!\left(\frac{p_2p_3-c_1}{p_3}\right)\right]\Bigg\}\ ,
\end{aligned}$$

with coefficients:

$$\begin{aligned}
c_0 &= \frac{\xi^2}{2T}\left(T + \frac{1}{\xi^2}\ln\!\left(\frac{\sigma(T)}{\sigma(0)}\right)\right)^2, \qquad c_1 = 4\xi^2\ , \\
p_0 &= 2\xi\sqrt{T} - \frac{\xi}{\sqrt{T}}\left(T + \frac{1}{\xi^2}\ln\!\left(\frac{\sigma(T)}{\sigma(0)}\right)\right), \qquad p_1 = \frac{2\xi}{\sqrt{T}}\ , \\
p_2 &= -\frac{\xi}{\sqrt{T}}\left(T + \frac{1}{\xi^2}\ln\!\left(\frac{\sigma(T)}{\sigma(0)}\right)\right), \qquad p_3 = \frac{4\xi}{\sqrt{T}}\ .
\end{aligned}$$

The first two conditional moments determine the lognormal proxy used for the inner integration, which can then be evaluated by a second Gaussian–Hermite quadrature. The resulting semi-analytical scheme delivers a smooth, accurate, and computationally efficient benchmark for SABR pricing.

2.5 Finite–difference scheme for SABR with general β

For the forward–measure SABR model with general elasticity parameter $\beta \in [0,1]$, the price $V = V(t, F, \sigma)$ of a forward call with strike K solves the two–dimensional parabolic PDE

$$\partial_t V = \tfrac{1}{2}\,\sigma^2 F^{2\beta}\,\partial_{FF} V + \tfrac{1}{2}\,\xi^2\sigma^2\,\partial_{\sigma\sigma} V + \rho\xi\sigma^2 F^{\beta}\,\partial_{F\sigma} V, \tag{1}$$

with terminal condition $V(T, F, \sigma) = \max(F - K, 0)$ and suitable boundary conditions in F and σ. We denote the three differential operators by

$$AV := \tfrac{1}{2}\,\sigma^2 F^{2\beta} V_{FF}, \qquad BV := \tfrac{1}{2}\,\xi^2\sigma^2 V_{\sigma\sigma}, \qquad CV := \rho\xi\sigma^2 F^{\beta} V_{F\sigma},$$

so that the PDE can be written compactly as

$$\partial_t V = (A + B + C)V.$$

Issue with the initial ADI scheme. In the first implementation we used a Douglas–type ADI step which, in discrete form, can be summarised as

$$\begin{aligned} A^n = A(V^n), \qquad B^n = B(V^n), \qquad C^n = C(V^n), \\ Y_0 = V^n + \Delta t\, C^n, \\ (I - \Delta t A)\, Y_1 = Y_0 + \Delta t\, A^n, \\ (I - \Delta t B)\, V^{n+1} = Y_1 + \Delta t\, B^n, \end{aligned}$$

where V^n approximates $V(t_n, \cdot, \cdot)$ with time step Δt. Formally expanding this scheme shows that the effective one–step operator is not simply

$$V^{n+1} \approx V^n + \Delta t(A + B + C)V^n + \mathcal{O}(\Delta t^2),$$

but instead contains additional $\mathcal{O}(\Delta t^2)$ terms proportional to A^2, B^2 and mixed products such as AB. Since both A and B are *diffusion* operators (second derivatives), these extra terms act as an artificial additional diffusion. In practice this manifests as systematically higher call prices and therefore a positive bias in the implied volatilities obtained from the finite–difference (FD) solver when compared to a reference such as the Hagan formula: the whole smile is effectively shifted upwards by several volatility points.

Craig–Sneyd ADI correction. To mitigate this bias we replaced the above scheme with a Craig–Sneyd (CS) ADI scheme in a similar fashion as [McGhee, 2020] (with $\theta = \frac{1}{2}$), which restores second–order accuracy in time and treats the mixed term C in a more balanced way. One time step of the CS scheme can be written as

$$\begin{aligned} A^n = A(V^n), \quad B^n = B(V^n), \quad C^n = C(V^n), \\ \text{(predictor)} \qquad Y_0 = V^n + \Delta t(A^n + B^n + C^n), \\ \text{(F–sweep)} \qquad (I - \theta\Delta t A)\, Y_1 = Y_0 - \theta\Delta t A^n, \\ (\sigma\text{–sweep}) \qquad (I - \theta\Delta t B)\, U_2 = Y_1 - \theta\Delta t B^n, \\ \text{(CS correction)} \qquad U_3 = U_2 + \tfrac{1}{2}\Delta t\big(C(U_2) - C^n\big), \\ \text{(second F–sweep)} \qquad (I - \theta\Delta t A)\, U_4 = U_3 - \theta\Delta t A^n, \\ \text{(second } \sigma\text{–sweep)} \qquad (I - \theta\Delta t B)\, V^{n+1} = U_4 - \theta\Delta t B^n, \end{aligned}$$

with $\theta = \frac{1}{2}$ in our implementation. This scheme is designed so that its local truncation error satisfies

$$V^{n+1} = V^n + \Delta t(A + B + C)V^n + \mathcal{O}(\Delta t^2),$$

with no systematic over-application of A and B. In particular, the CS correction term $\frac{1}{2}\Delta t(C(U_2) - C^n)$ upgrades the mixed derivative contribution from a purely explicit first–order treatment to a second–order consistent one. Numerically this removes the artificial extra diffusion present in the original scheme and significantly reduces the bias in the FD implied volatilities relative to the Hagan approximation.

Practical note. Re-running the entire fine-tuning calibration and neural–network training with the corrected CS–ADI solver would require substantial computational resources (several hours on a multi-core machine). For this reason, we did not recompile *all* the numerical experiments within the scope of this project. However, the updated and tested implementation of the CS–ADI scheme for general β is available in the accompanying Git repository, and can be used to regenerate the finite–difference reference dataset if more accurate ground truth is required.

3 Methodology

This section describes the computational workflow adopted in this study. All code used to generate datasets, train neural networks, and produce the figures shown in this report is publicly available in the project repository. The repository follows a modular structure: the `src/` directory contains the reusable core modules (SABR engines, strike-grid construction, dataset generators, neural-network architectures, learning-rate schedulers, and plotting utilities), while the top-level scripts implement the different experiments performed in Section 4.

3.1 Source code structure (`src/`)

The `src/` directory contains all reusable, vectorized modules employed throughout the study. Whenever relevant, routines are implemented in two variants: a baseline version for the lognormal case $\beta = 1$, and a generalized version applicable to arbitrary elasticity parameters $\beta \neq 1$. Below we summarise the purpose of each module.

Strike–grid construction (`MaxK_minK.py`). This module constructs the 10–node strike grid used throughout the work. Given

$$\psi_{\ln}(T) = \frac{\sigma_0^2}{\xi^2}\big(e^{\xi^2 T} - 1\big), \qquad \chi(T) = \sqrt{\frac{e^{\xi^2 T} - 1}{\xi^2 T}},$$

the function

$$\texttt{strike\_ratio}(F_0, \sigma_0, \rho, \xi, T, \eta_S, \eta_\sigma) = \exp\Big(-\tfrac{1}{2}\psi_{\ln}(T) + \sqrt{\psi_{\ln}(T)}\,\big[\eta_S + \rho\,\eta_\sigma\,\chi(T)\big]\Big)$$

returns the ratio K/F_0. With $\eta_S \in [-3.5, 3.5]$ and $\eta_\sigma = 1.5$, the resulting interval

$$K_{\min} = F_0\,\texttt{strike\_ratio}(-3.5), \qquad K_{\max} = F_0\,\texttt{strike\_ratio}(+3.5)$$

is discretised into ten uniformly spaced log-moneyness nodes:

$$\big[\log(K_{\min}/F_0),\, \log(K_{\max}/F_0)\big].$$

This produces a stable and economically meaningful strike grid for all dataset generators.

SABR implied-volatility engines. The module `sabr_hagan.py` implements the Hagan approximation for $\beta = 1$. The companion module `sabr_hagan_different_betas.py` extends the approximation to the elasticity regime $0 \leq \beta < 1$.

Conditional-integration engine (`sabr_integrationF.py`). The semi-analytical conditional integration approach of [McGhee, 2011] is implemented in the module `sabr_integrationF.py`. The routine computes SABR option prices by integrating the conditional distribution of the forward under lognormal volatility using nested Gaussian–Hermite quadratures, followed by numerical inversion to implied volatility.

Finite-difference SABR engine (`sabr_fd_beta.py`). This module implements a two-factor finite-difference solver for the SABR model with a general elasticity parameter $0 \leq \beta \leq 1$. It provides the finite-difference ground truth used to construct the 2FDM datasets that feed the fine-tuning stage of the neural network.

Dataset generation (`data_vector*.py`). These modules construct the training and validation datasets used throughout the experiments. Both follow the same pipeline:

1. sample (T, σ_0, ξ, ρ) from the prescribed SABR domain;
2. build ten log-moneyness nodes via `MaxK_minK.py`;
3. evaluate Hagan implied volatilities at each node;
4. standardize inputs and targets using training-set statistics.

The generalized version additionally stores *per-β* normalization parameters to ensure numerical consistency across the elasticity sweep.

Neural–network architecture (`nn_arch.py`). This module defines *GlobalSmileNetVector*, a feed-forward network with one hidden layer (Softplus activation). The input vector

$$(T,\, \sigma_0,\, \xi,\, \rho,\, x_1, \ldots, x_{10})$$

is mapped to ten standardized implied volatilities

$$\big(\sigma_{\text{imp}}(K_1), \ldots, \sigma_{\text{imp}}(K_{10})\big).$$

The architecture mirrors the structure of [McGhee, 2020] and provides a smooth surrogate for the full volatility smile.

Learning–rate scheduler (`lr_schedules.py`). This module implements a multi-stage adaptive learning-rate policy combining exponential decay, EMA-smoothed validation monitoring, patience-controlled *bump* restarts, and a late-stage secondary decay regime. This yields stable convergence across network widths and across values of β.

Plotting routines. The modules, for $\beta = 1$, `plotting_vector.py` and, for $0 \leq \beta < 1$, `plotting_vector_different_betas.py` generate all diagnostic figures used in Section 4. Each plot contains (i) the ANN-implied volatility smile compared with the Hagan formula and (ii) the corresponding pointwise error across strikes.

3.2 Neural-network training on the Hagan approximation

Neural networks are trained to reproduce the implied-volatility mapping generated by the Hagan asymptotic expansion [Hagan et al., 2002], following the experimental design of [McGhee, 2020]. The model learns the standardized mapping

$$(T, \sigma_0, \xi, \rho, \ln(K/F)) \longmapsto \sigma_{\text{imp}},$$

over the parameter domain

$$T \in [1\text{D}, 2\text{Y}], \qquad \sigma_0 \in [5\%, 50\%], \qquad \rho \in [-0.9, 0.9], \qquad \xi(T) \in [5\%, 400\%]\sqrt{\tfrac{1\text{M}}{T}}.$$

Each volatility smile is evaluated at ten uniformly spaced log-moneyness nodes, and both inputs and targets are standardized using statistics computed from the training set. As in [McGhee, 2020], short and long maturities ($T \leq 1$ year vs. $T > 1$ year) are trained using two separate models to improve numerical stability.

Training is performed with mini-batches using the Adam optimizer. At each epoch:

1. model parameters are updated on shuffled training batches,
2. validation loss is evaluated on a held-out dataset,
3. the learning rate is updated by the *PaperStyleLR* scheduler.

The scheduler combines exponential decay, validation-driven "bump" restarts, and a late-stage decay phase, and effectively implements regularised early stopping by retaining the checkpoint with the lowest validation error.

Baseline case: $\beta = 1$ (global_all_in_one_phase1.py). The first experiment trains the network exclusively on the lognormal Hagan approximation. This serves as a reference implementation for the classical SABR specification and provides a direct benchmark against the results reported in [McGhee, 2020].

Elasticity sweep with β as input (global_all_in_one_phase1_beta_input.py). To analyse the behaviour of the ANN across the full elasticity spectrum, we additionally train a model family in which β is treated as an explicit input feature. In the code, training and validation datasets are constructed by sampling $(T, \sigma_0, \xi, \rho, \beta)$ over the usual SABR domain, with β drawn continuously from the interval $[0, 1]$. Each standardized input vector therefore has the form

$$(T, \sigma_0, \xi, \rho, \beta, x_1, \ldots, x_{10}) \in \mathbb{R}^{15},$$

and the targets remain the ten Hagan implied volatilities evaluated at the fixed strike grid. This produces a set of multi-β surrogates that learn a single smooth mapping across the entire elasticity range $\beta \in [0, 1]$.

Hidden-layer width ablation (global_all_in_one_phase_different_width.py). To study representational capacity, the width of the single hidden layer is varied systematically across two regimes:

$$\text{Small–to–medium: } W \in \{16, 32, 64, 128, 256\},$$
$$\text{Medium–to–large: } W \in \{256, 500, 1000, 2000, 3000\}.$$

For each width, independent models are trained for short and long maturities under identical sampling, optimization, and scheduling conditions. This ablation study identifies the minimal architecture capable of reproducing the analytical Hagan surface at high accuracy and characterises how performance improves with increasing network capacity.

3.3 Integration-based method

In `global_all_in_one_phase2_integration.py`, we implemented the fine-tuning phase for the conditional integration function. Here, the target surface is the conditional-integration engine described in Section 3.1, as wrapped by `src/data_vector_integration.py`. To start, we load a precomputed cache (`datasets/phase2_integration_paper.npz`), which contains standardized inputs and integration-based implied volatilities for both training and validation:

$$(X_{\text{tr}}^{\text{raw}}, Y_{\text{tr}}^{\text{true}}, X_{\text{va}}^{\text{raw}}, Y_{\text{va}}^{\text{true}}, \text{meta}).$$

The raw feature vectors have the same structure as in the Hagan training (maturity, SABR parameters and smile nodes), and the labels Y^{true} are implied volatilities computed through the use of `sabr_integrationF.py`. Following the convention of [McGhee, 2020], the script splits the raw data into short- and long-maturity buckets at $T = 1\,\text{year}$, and computes mean and standard deviation from the short-maturity training set. Once the dataset for training and validation is loaded, we train a family of *GlobalSmileNetVector* networks with the same architecture as in the first training, but using the integration-based volatilities as targets. For each hidden-layer width $W \in \{250, 500, 750, 1000\}$ and for each maturity regime (regime $\in \{\text{short}, \text{long}\}$), a separate model is trained with MSE loss, Adam optimizer, and the *PaperStyleLR* scheduler:

$$\theta \;\mapsto\; \mathcal{L}_{\text{P2}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left\|\hat{\sigma}_{\text{ANN}}(x_i;\theta) - \sigma_{\text{int}}(x_i)\right\|^2.$$

In order to train the model we compute a warm start using the corresponding results from the Hagan training. The best-performing model is kept for each (W, regime) pair and saved under `night_runs/phase2_integration_run/w`*W*`_`*regime*`/best_w`*W*`.pt`, together with the associated scaler file.

3.4 Finite-difference scheme method

In the script `global_all_in_one_phase2_beta_input.py`, we implement the finite-difference refinement across the full elasticity range, which fine-tunes the Hagan-based β-input networks on implied volatilities generated by the 2FDM solver. The function `load_phase2_fd_beta_cached()` provides a cached Phase 2 dataset `phase2_fd_beta_input_big.npz`, containing inputs of the form

$$(T, \sigma_0, \xi, \rho, \beta, x_1, \ldots, x_{10}) \in \mathbb{R}^{15},$$

and corresponding FD-based implied volatilities for training and validation. As we did in the first training, the standardised data are split into short and long maturities at $T = 1\,\text{year}$, and separate models are trained for each subset. For every hidden-layer width $W \in \{250, 500, 750, 1000\}$, the script trains a *GlobalSmileNetVector* with input dimension 15 and output dimension 10, using mean-squared error loss, the Adam optimizer, and the *PaperStyleLR* scheduler with width-specific hyperparameters. Moreover, as we did for the conditional integration case, we use the results from the first training on the Hagan approximation (stored in `night_runs/phase1_beta_input_run/w`*W*`_`*regime*`/best_w`*W*`.pt`) as a warm start for the fine-tuning on the 2FDM method. In this way, the fine tuning learning refines an already well-trained Hagan surrogate.

4 Results

This section presents the empirical results obtained from the neural-network experiments across the analytical, semi-analytical, and numerical SABR frameworks. We begin by analysing the neural network trained on the lognormal Hagan approximation ($\beta = 1$), then extend the analysis

to the generalized elasticity case ($\beta \in [0, 1]$) and to the width-ablation study. We then examine how the fine-tuning improves the surrogate accuracy when trained on finite-difference data, and finally compare these results with those based on the conditional-integration approach.

4.1 Lognormal baseline ($\beta = 1$)

Figures 1–3 display the neural-network approximation for the lognormal SABR model ($\beta = 1$) under three representative parameter configurations: a short-maturity case ($T = 14\,\text{days}$), an intermediate case ($T = 6\,\text{months}$), and a long-maturity case ($T = 1\,\text{year}$). In each figure, panel (a) compares the ANN-predicted volatility smile with the analytical Hagan approximation, while panel (b) shows the corresponding percentage error across strikes.

Short maturity (Figure 1). For the very short-term configuration ($T = 14\,\text{days}$, $\sigma_0 = 30\%$, $\xi = 160\%$, $\rho = -60\%$), the volatility smile exhibits pronounced negative skew due to the high vol-of-vol and negative correlation. Across all hidden-layer widths, the ANN closely tracks the steep downward curvature, with pointwise errors typically below 0.2% over the entire moneyness range. Residual fluctuations in panel (b) are small and approximately symmetric, indicating that the near-ATM curvature is captured without inducing numerical instabilities.

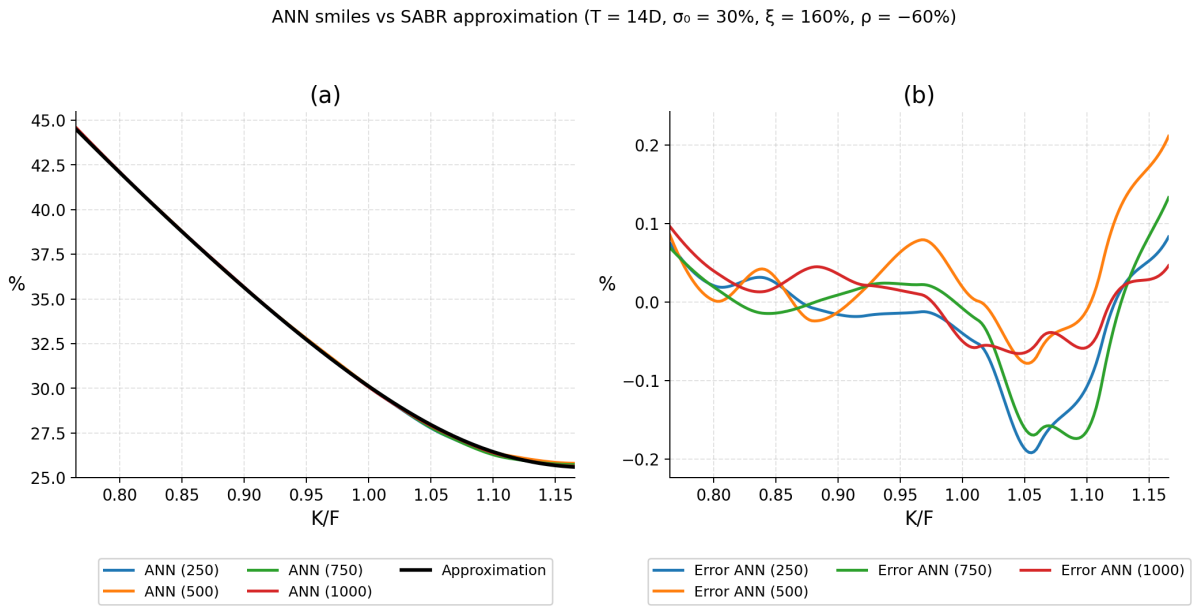

Figure 1: Short-maturity regime ($T = 14\,\text{days}$, $\sigma_0 = 30\%$, $\xi = 160\%$, $\rho = -60\%$). Left: ANN-predicted volatility smiles vs. Hagan approximation. Right: corresponding percentage errors across strikes.

Intermediate maturity (Figure 2). At $T = 6\,\text{months}$, with $\sigma_0 = 30\%$, $\xi = 40\%$ and $\rho = 0\%$, the smile becomes more symmetric and less skewed. The ANN again reproduces the analytical surface with high accuracy: the mean absolute error is below 0.05%, and the error curves in panel (b) display only mild oscillations in the wings, without evidence of systematic bias. This suggests that the network adapts smoothly to changes in maturity and smile shape.

Long maturity (Figure 3). For the long-term configuration ($T = 1\,\text{year}$, $\sigma_0 = 20\%$, $\xi = 30\%$, $\rho = +30\%$), the smile develops the characteristic upward curvature associated with positive correlation. Panel (a) shows that the ANN closely matches this shape, including the mild convexity at high strikes. The errors in panel (b) remain very small (typically within $\pm 0.05\%$), with slightly larger deviations only near the boundaries of the sampled strike range.

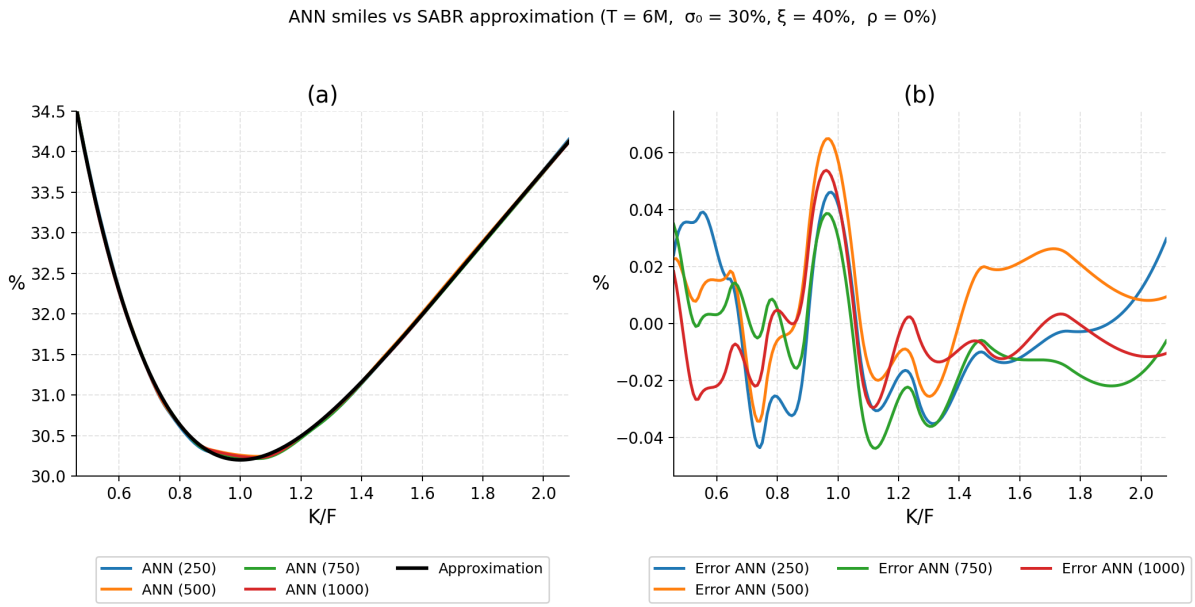

Figure 2: Intermediate-maturity regime ($T = 6\,\text{months}$, $\sigma_0 = 30\%$, $\xi = 40\%$, $\rho = 0\%$). Left: ANN-predicted volatility smiles vs. Hagan approximation. Right: corresponding percentage errors across strikes.

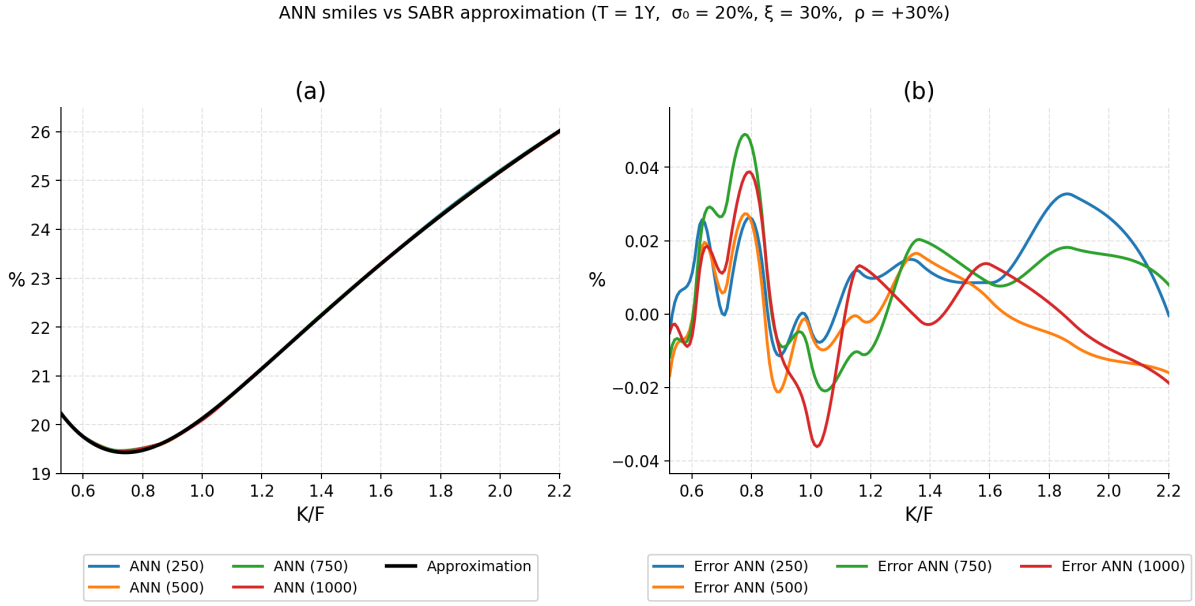

Figure 3: Long-maturity regime ($T = 1\,\text{year}$, $\sigma_0 = 20\%$, $\xi = 30\%$, $\rho = +30\%$). Left: ANN-predicted volatility smiles vs. Hagan approximation. Right: corresponding percentage errors across strikes.

Comparison across maturities. Taken together, Figures 1–3 show that the neural network reproduces the Hagan approximation with errors well below one basis point for the vast majority of strikes. Short maturities display stronger skew and higher sensitivity to correlation, whereas longer maturities yield smoother, less asymmetric smiles; in all cases, the ANN maintains a comparable level of precision. The $\beta = 1$ configuration therefore provides a robust benchmark for the subsequent elasticity and architecture experiments.

4.2 Generalized elasticity case ($\beta \in [0, 1]$)

Figures 4–7 display the implied–volatility surface as a function of moneyness and elasticity parameter β, together with the pointwise error between the ANN surrogate and the Hagan implied–volatility formula. These visualisations allow us to assess how approximation accuracy evolves as both network capacity and the elasticity parameter vary.

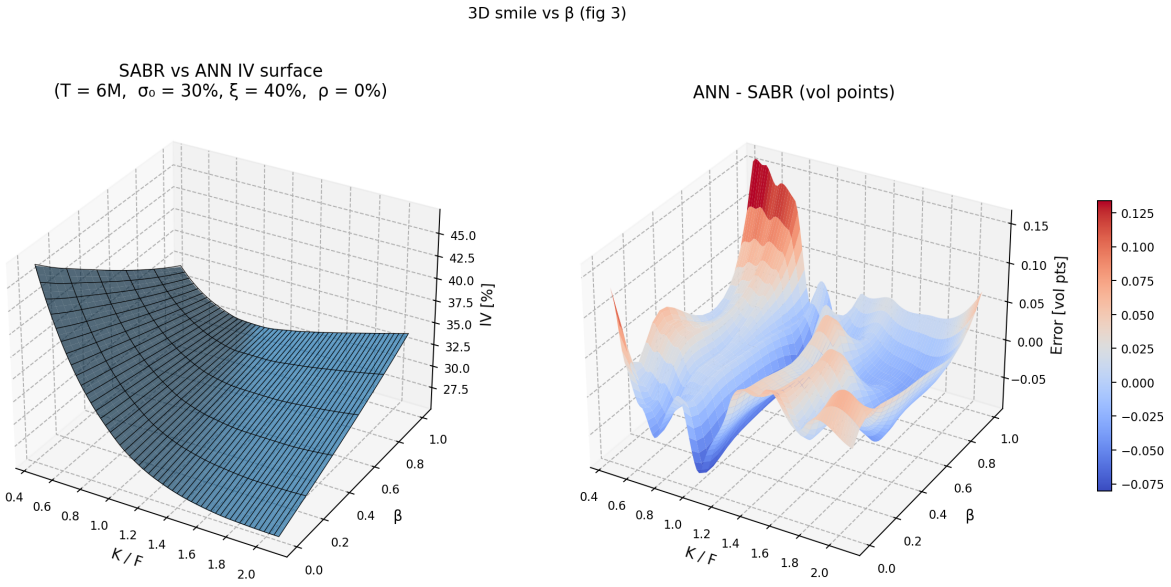

Figure 4: Generalized elasticity case ($\beta \in [0, 1]$), network width $W = 250$. Left: ANN-predicted implied-volatility surface vs. Hagan formula. Right: corresponding pointwise percentage errors across $(K/F, \beta)$.

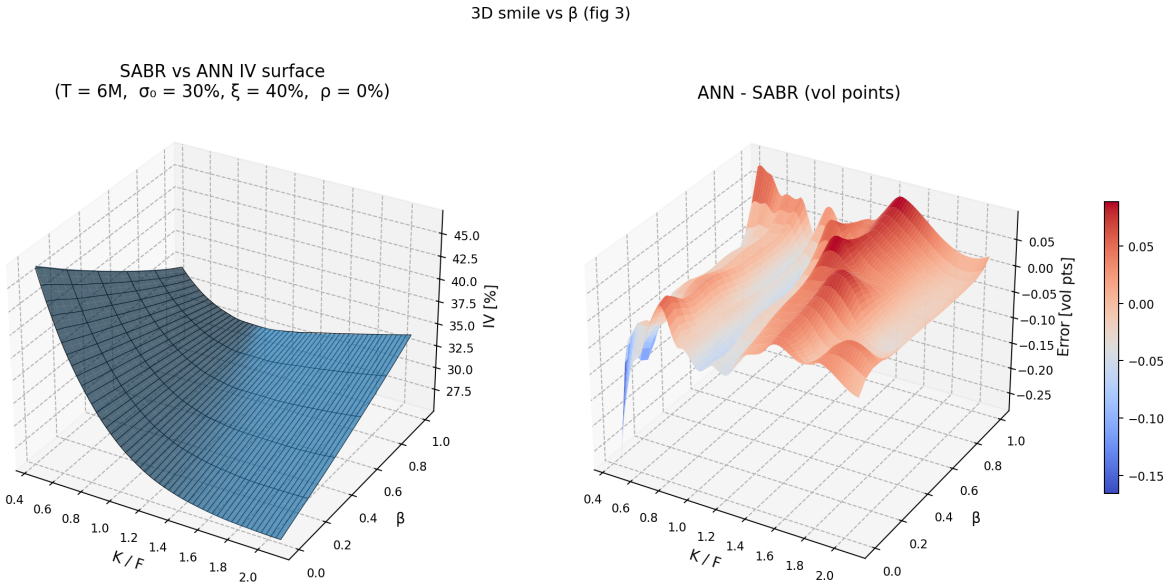

Figure 5: Generalized elasticity case ($\beta \in [0, 1]$), network width $W = 500$. Left: ANN-predicted implied-volatility surface vs. Hagan formula. Right: corresponding pointwise percentage errors across $(K/F, \beta)$.

Across all architectures, the ANN successfully reproduces the global geometry of the Hagan surface over the full elasticity range $\beta \in [0, 1]$. However, for small widths (e.g. $W = 250$), the

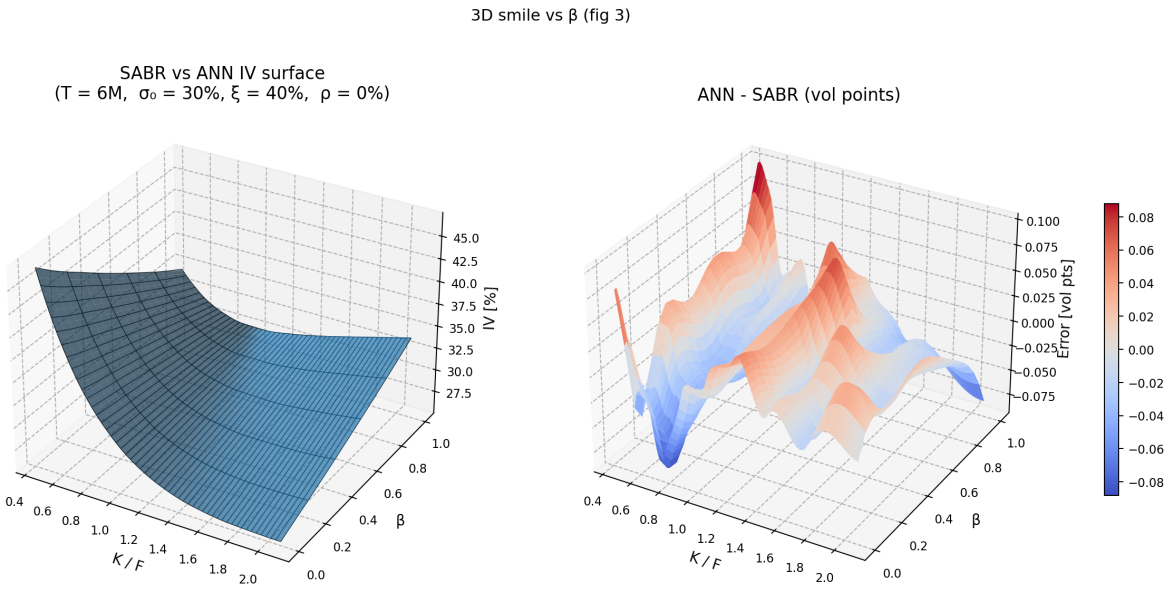

Figure 6: Generalized elasticity case ($\beta \in [0, 1]$), network width $W = 750$. Left: ANN-predicted implied-volatility surface vs. Hagan formula. Right: corresponding pointwise percentage errors across $(K/F, \beta)$.

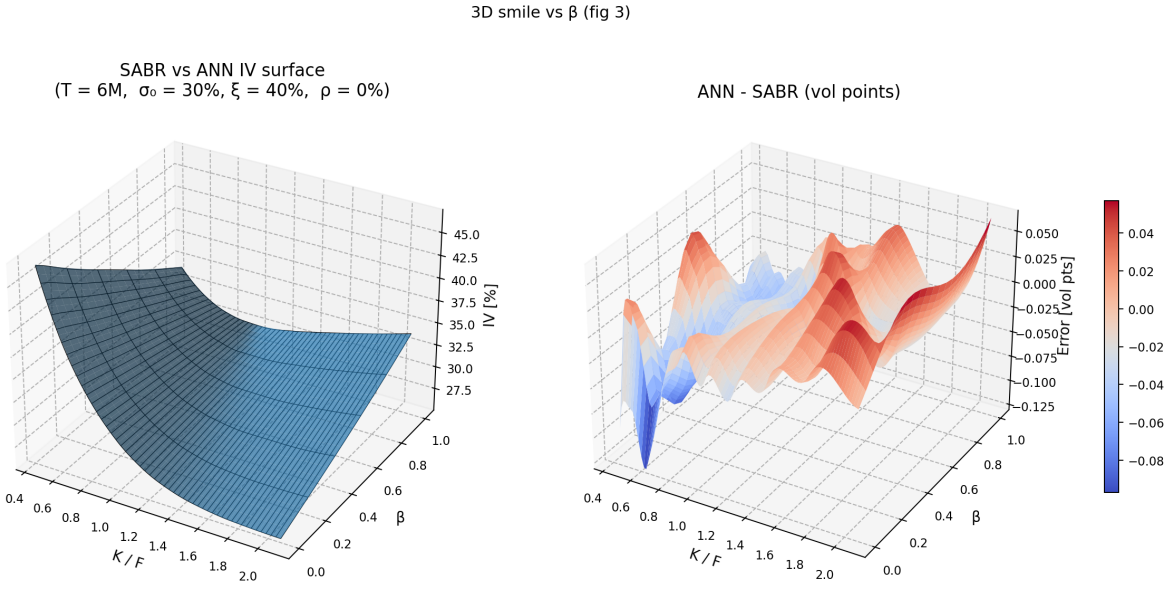

Figure 7: Generalized elasticity case ($\beta \in [0, 1]$), network width $W = 1000$. Left: ANN-predicted implied-volatility surface vs. Hagan formula. Right: corresponding pointwise percentage errors across $(K/F, \beta)$.

error surface displays larger oscillations, particularly in the wings and at the extremes of the β domain. As the width increases, the approximation improves significantly: the $W = 750$ and $W = 1000$ networks produce small, smooth error surfaces concentrated around zero, indicating stable generalisation across both strike and elasticity dimensions.

4.3 Hidden-layer width investigation

We compare neural networks trained on the Hagan approximation across a wide range of hidden-layer widths. Figures 8–13 present, for each maturity regime, the ANN-predicted volatility smiles together with the corresponding percentage errors for different widths. Small-width networks reproduce the overall shape of the smile but show clear biases and larger oscillations, especially in the wings. Medium widths already deliver very accurate fits, with errors centred around zero and significantly reduced variability. Moreover, large widths yield only marginal improvements, indicating diminishing returns once the network capacity is sufficient to match the smooth analytical structure of the Hagan surface.

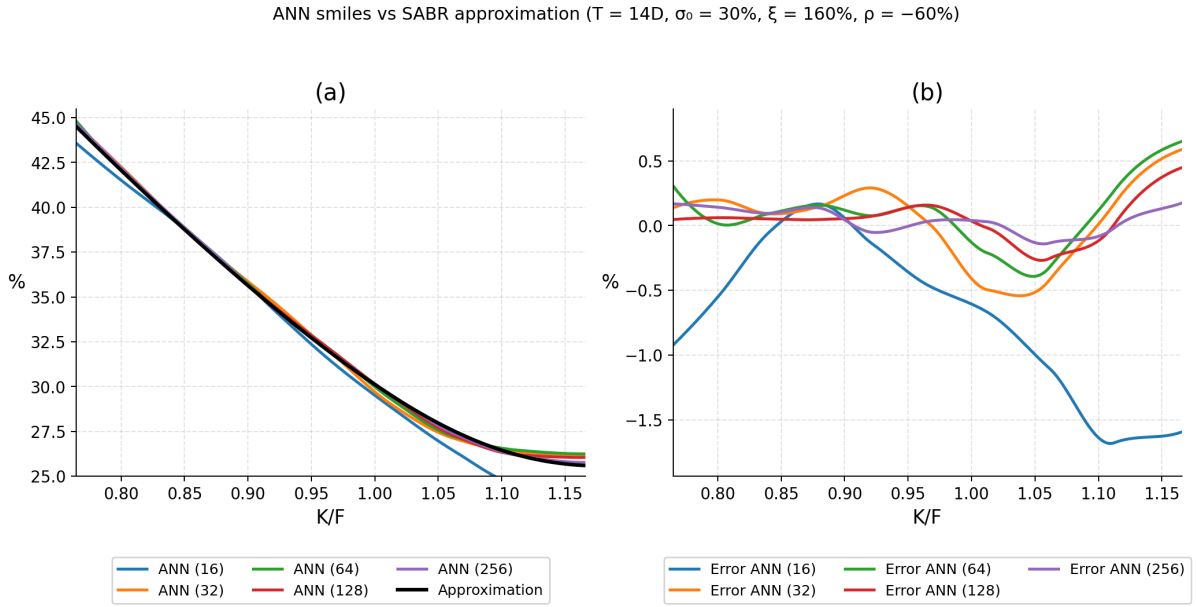

Figure 8: Short-maturity regime ($T = 14$ days): ANN smiles and errors for small and medium hidden-layer widths.

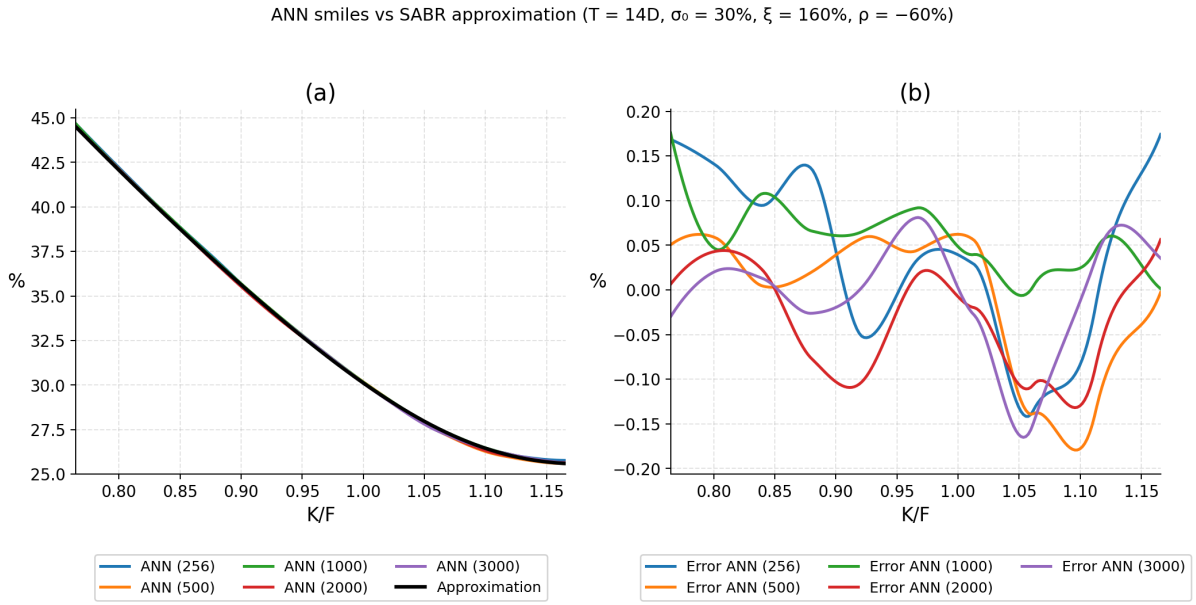

Figure 9: Short-maturity regime ($T = 14$ days): ANN smiles and errors for medium and large hidden-layer widths.

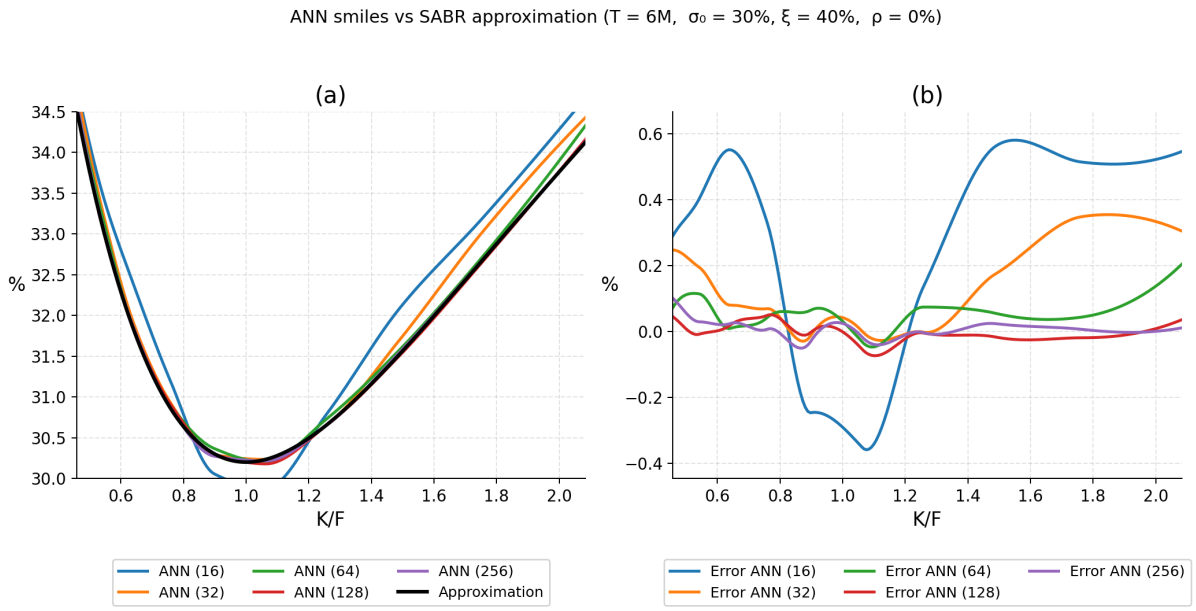

Figure 10: Intermediate-maturity regime ($T = 6$ months): ANN smiles and errors for small and medium hidden-layer widths.

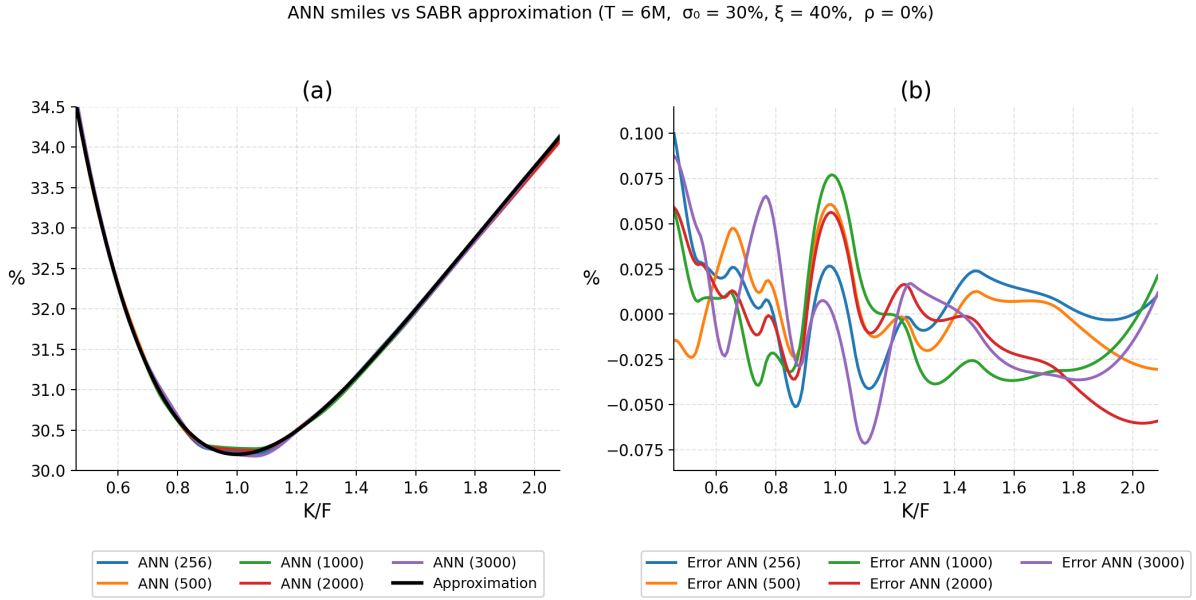

Figure 11: Intermediate-maturity regime ($T = 6$ months): ANN smiles and errors for medium and large hidden-layer widths.

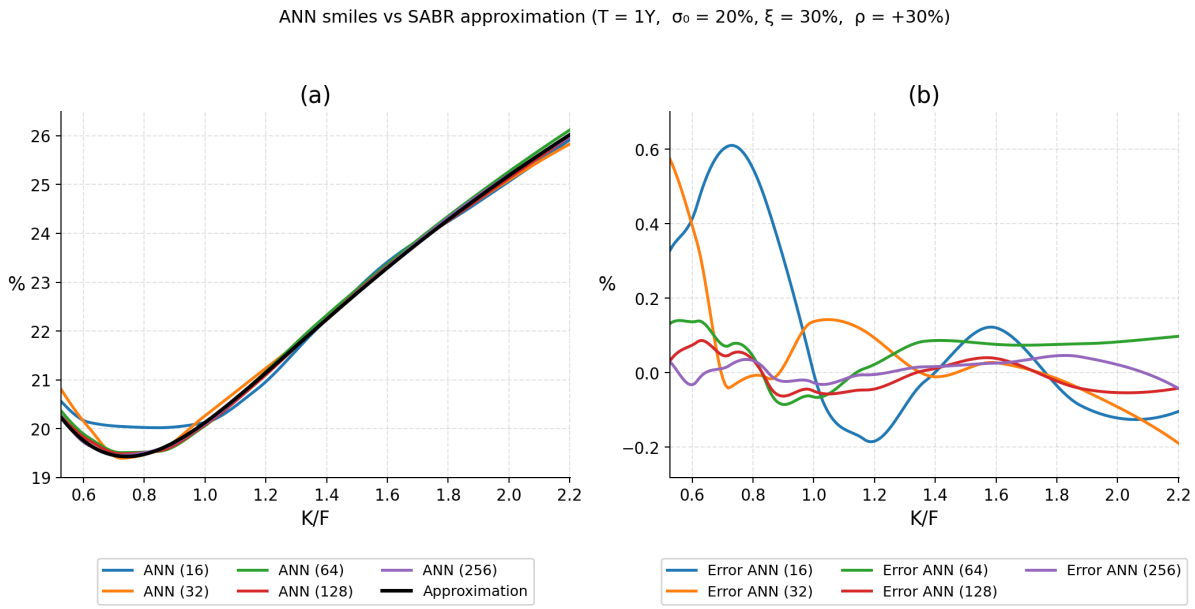

Figure 12: Long-maturity regime ($T = 1$ year): ANN smiles and errors for small and medium hidden-layer widths.

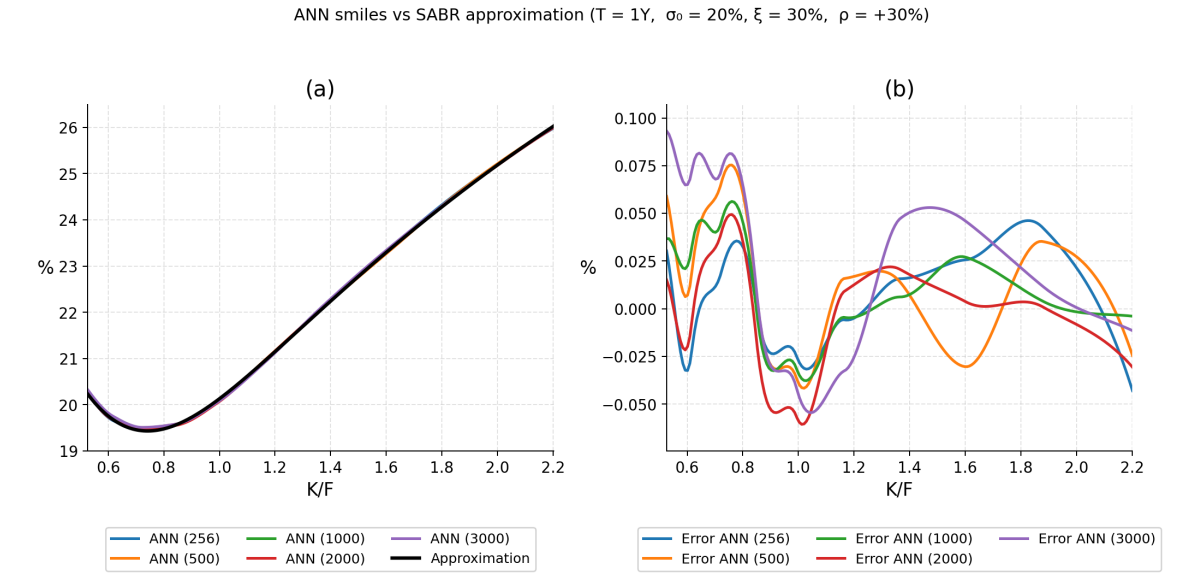

Figure 13: Long-maturity regime ($T = 1$ year): ANN smiles and errors for medium and large hidden-layer widths.

4.4 Integration-based method

Figures 14–16 evaluate the performance of the neural-network surrogate when trained on implied volatilities generated by the conditional-integration method of [McGhee, 2011]. Unlike the Hagan approximation, which is closed-form, and unlike the 2FDM solver, which relies on a full PDE discretisation, the integration-based approach produces implied volatilities by numerically integrating the conditional distribution of the forward process under the SABR dynamics. Although this method is substantially lighter than 2FDM in terms of computational cost, its numerical structure introduces several sources of irregularity that make the fine-tuning on high-fidelity data less effective. In particular, the integration solver: (i) exhibits higher sensitivity

to the integration grid and truncation limits; (ii) produces option values with higher pointwise variance relative to the smooth 2FDM surface; and (iii) is not globally smooth in all parameters, especially in regions where the SABR transition density becomes sharply skewed. These properties propagate into the implied-volatility surface and reduce the regularity of the training targets. Since the neural architecture of the fine-tuning is designed under the assumption of a smooth, coherent volatility surface (as produced by 2FDM), the surrogate struggles to fully reproduce the integration-based prices. This explains the larger oscillations, increased error magnitudes, and less systematic improvement across architectures observed in Figures 14–16.

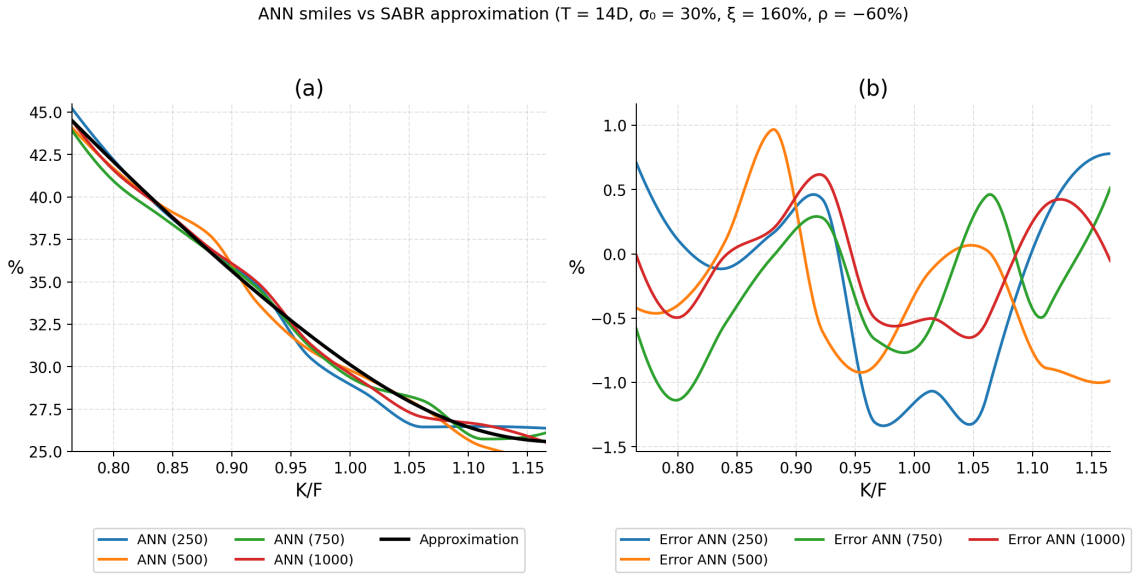

Figure 14: Short-maturity regime ($T = 14\,\text{days}$, $\sigma_0 = 30\%$, $\xi = 160\%$, $\rho = -60\%$). Left: ANN-predicted volatility smiles vs. integration-based values. Right: corresponding percentage errors across strikes.

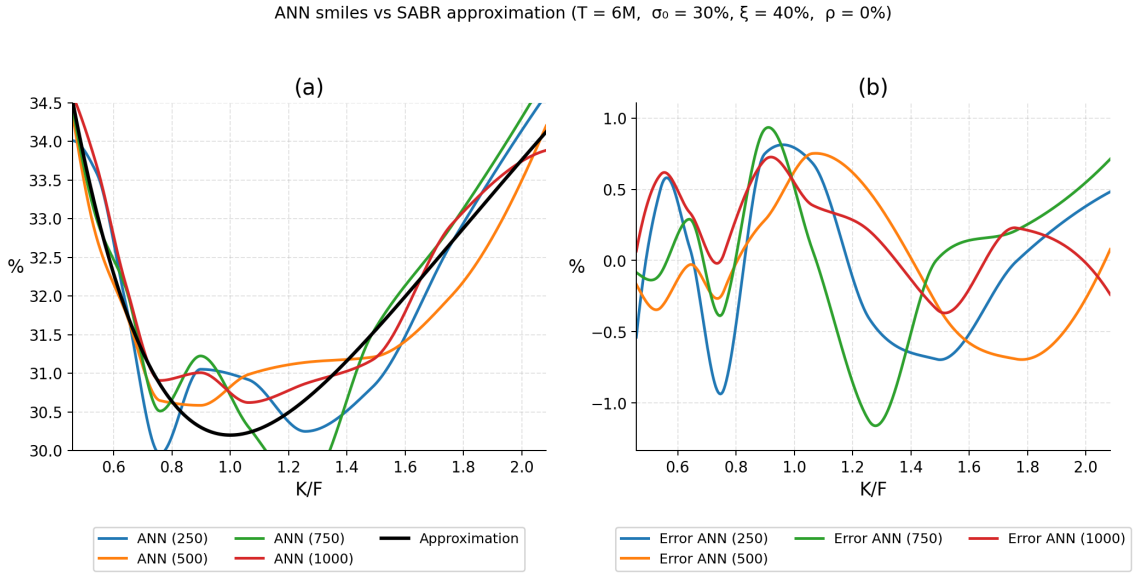

Figure 15: Intermediate-maturity regime ($T = 6\,\text{months}$, $\sigma_0 = 30\%$, $\xi = 40\%$, $\rho = 0\%$). Left: ANN-predicted volatility smiles vs. integration-based values. Right: corresponding percentage errors across strikes.

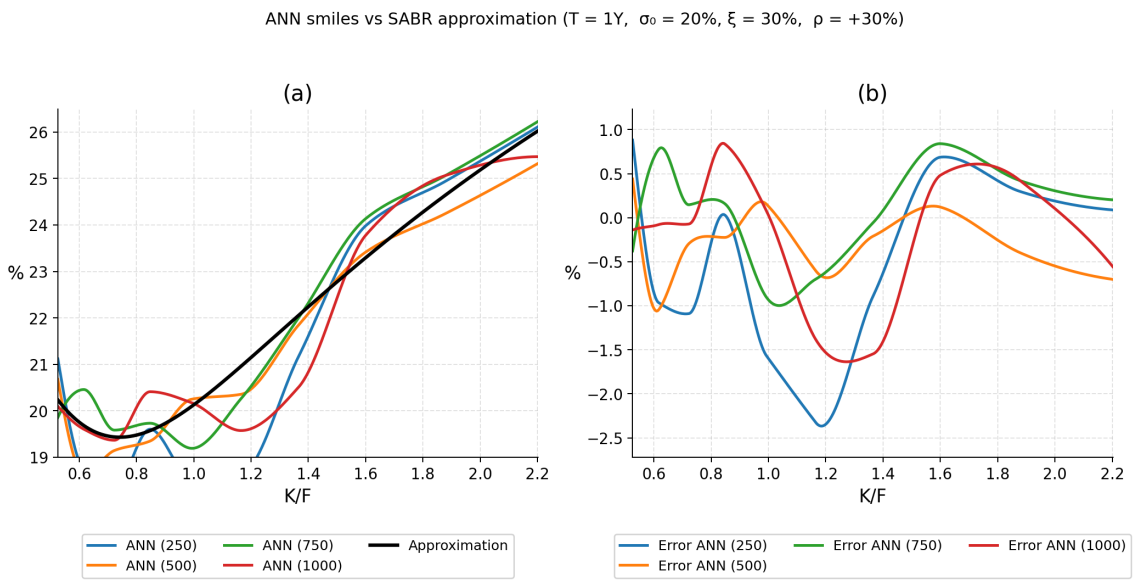

Figure 16: Long-maturity regime ($T = 1\,\text{year}$, $\sigma_0 = 20\%$, $\xi = 30\%$, $\rho = +30\%$). Left: ANN-predicted volatility smiles vs. integration-based values. Right: corresponding percentage errors across strikes.

4.5 Finite-difference scheme method

Figures 17–20 report the behaviour of the neural network when fine-tuned on implied volatilities generated by the two-factor finite-difference (2FDM) solver. Each figure shows, for a representative maturity and across the full elasticity range $\beta \in [0, 1]$: (i) the finite-difference implied-volatility surface obtained from the ADI scheme, (ii) the pointwise ANN–FD errors, and (iii) the corresponding Hagan–FD errors, which provide a baseline for comparison.

Observed behaviour. Across all widths, the ANN is able to learn the global geometry of the finite-difference volatility surface. The correction over the Hagan approximation is clearly visible: for every value of β, the ANN reduces the systematic Hagan bias and captures the curvature induced by the PDE solution, especially in the wings and for intermediate elasticity values. Moreover, as expected, larger widths improve stability and produce smoother error surfaces. The $W = 750$ and $W = 1000$ models exhibit more consistent ANN–FD alignment and visibly lower residual variance. The behaviour therefore mirrors what was observed in the Hagan-only experiments: once the network is sufficiently wide, incremental gains become modest.

Note on the 2FDM surfaces. The finite-difference surfaces displayed here were generated with the original version of the FD solver. During the final stage of the project, we identified a shift arising from a grid-alignment issue in the older implementation. A corrected version of the solver has now been implemented, and preliminary checks confirm that the updated FD surfaces are globally smoother and that the ANN refinement benefits correspondingly. Due to time constraints, the full recomputation of the results for the 2FDM is left as future work; however, the qualitative conclusions drawn from the existing experiments remain valid: large-width ANNs consistently outperform the Hagan approximation and track the numerical solver with high fidelity across the entire elasticity range.[1]

[1]See Section 3 for a detailed explanation of the ADI correction and its numerical impact.

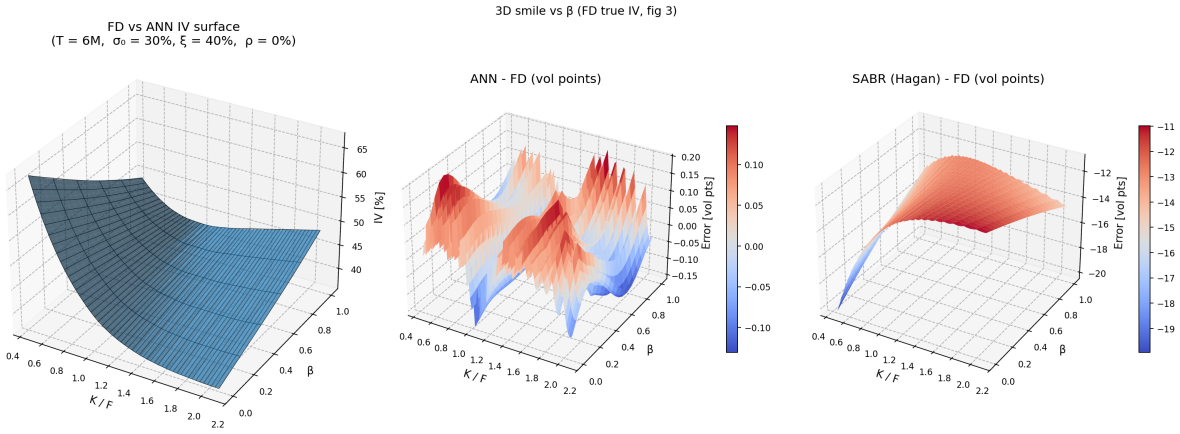

Figure 17: Generalized elasticity case ($\beta \in [0, 1]$), FD training target, network width $W = 250$. Left: finite-difference implied-volatility surface. Centre: ANN–FD pointwise errors. Right: Hagan–FD pointwise errors.

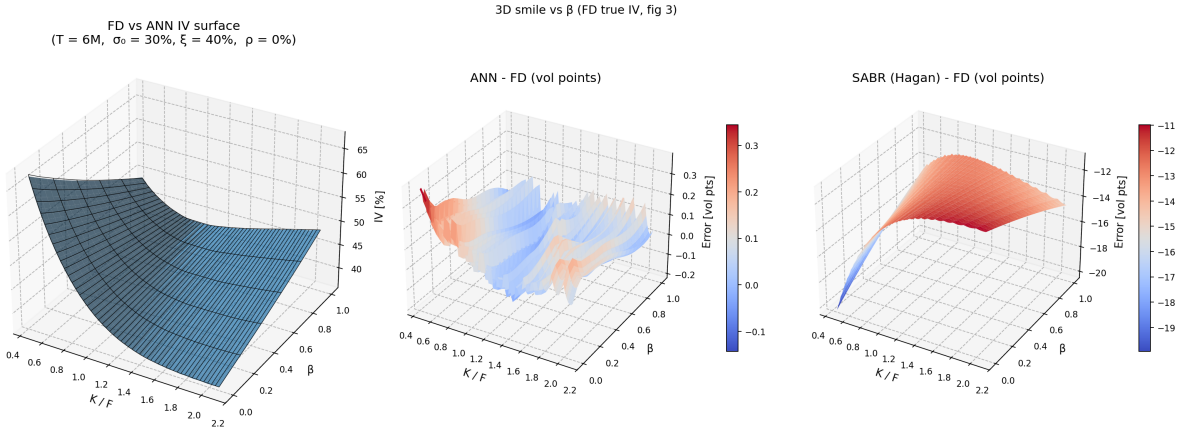

Figure 18: Generalized elasticity case ($\beta \in [0, 1]$), FD training target, network width $W = 500$. Left: finite-difference implied-volatility surface. Centre: ANN–FD pointwise errors. Right: Hagan–FD pointwise errors.

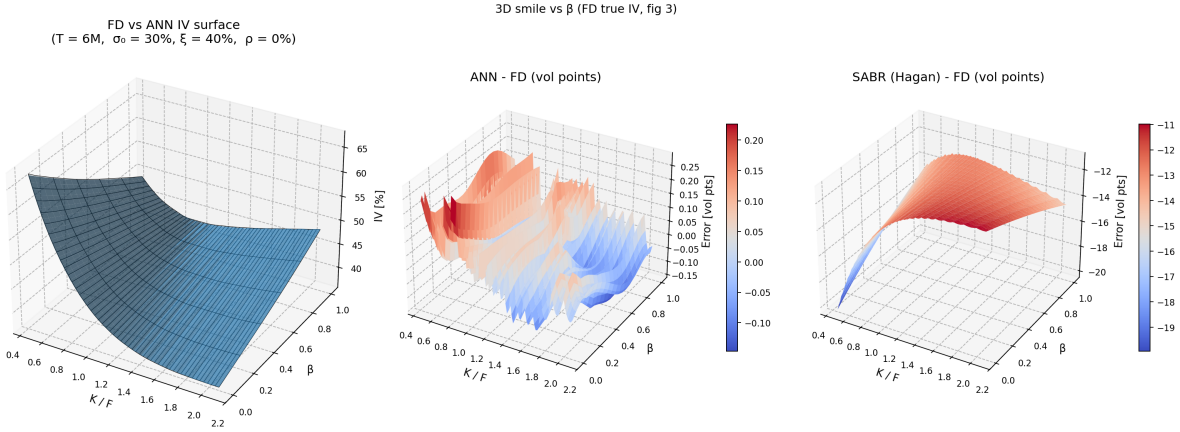

Figure 19: Generalized elasticity case ($\beta \in [0, 1]$), FD training target, network width $W = 750$. Left: finite-difference implied-volatility surface. Centre: ANN–FD pointwise errors. Right: Hagan–FD pointwise errors.

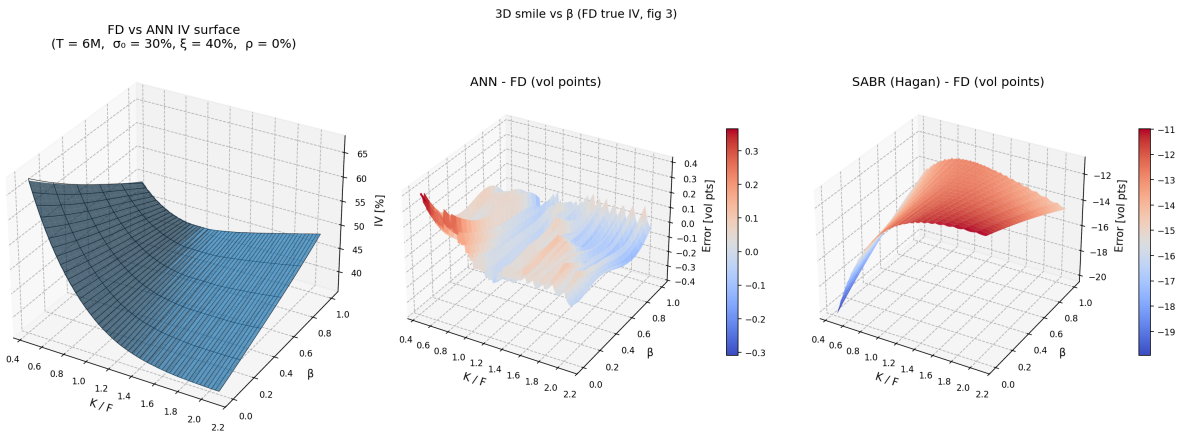

Figure 20: Generalized elasticity case ($\beta \in [0, 1]$), FD training target, network width $W = 1000$. Left: finite-difference implied-volatility surface. Centre: ANN–FD pointwise errors. Right: Hagan–FD pointwise errors.

Overall assessment. Despite the solver-related caveat, the results demonstrate that the 2FDM-based refinement phase behaves as expected: the ANN learns coherent corrections to the analytical baseline, and wider architectures achieve lower and more homogeneous ANN–FD errors. The finite-difference solver therefore remains the most reliable high-fidelity reference for SABR implied-volatility surfaces.

5 Discussion and Interpretation

5.1 Neural Network on the Hagan approximation function

The experiments executed on the Hagan implied-volatility approximation allow for a precise conclusion on how well shallow neural networks are able to reproduce the analytical volatility surfaces derived from the SABR asymptotic expansion. Taken together, the three experimental components (the lognormal baseline ($\beta = 1$), the elasticity sweep ($\beta < 1$), and the width-ablation study) yield a consistent and interpretable set of findings.

Model accuracy for the lognormal case ($\beta = 1$). The network succeeds in reproducing the Hagan implied-volatility surface with very high precision. For representative short-, medium-, and long-maturity configurations, the predicted smiles align almost perfectly with their analytical counterparts, typically within a fraction of a basis point. The model captures both the ATM curvature and the wing behaviour without introducing oscillations or artifacts. This confirms that, on the compact domain used for training, the Hagan formula is sufficiently regular to be learned by a shallow feed-forward architecture.

Behaviour across elasticity parameters ($\beta < 1$). Repeating the same training procedure for $\beta \in [0, 1]$ shows that the neural surrogate generalizes robustly across the entire elasticity range. The implied-volatility smiles transition smoothly from the strongly skewed normal regime ($\beta = 0$) to the milder lognormal regime ($\beta = 1$), and the network accurately tracks this evolution for all the maturities we tested. Moreover, the approximation error remains small and stable across strikes, indicating that the network has learned not only the local shape of the volatility surface but also the global dependence of the asymptotics on β. This is consistent with SABR theory: as β increases, the skew is progressively dampened, and the neural approximation captures this trend without retraining instability or parameter sensitivity.

Effect of network capacity. The width-ablation analysis we performed reveals a clear capacity–accuracy relationship. Networks with very small hidden layers ($W \leq 64$) tend to underfit, most visibly in the smile wings and in short-maturity configurations where curvature is stronger. Performance improves rapidly as width increases, and models with $W \geq 500$ achieve near-perfect agreement with the analytical surface. Beyond $W \approx 750$–1000, gains become negligible, suggesting that the Hagan mapping can be well approximated by relatively compact architectures. This aligns with theoretical expectations: the implied-volatility function is smooth across the interior of the domain, and therefore does not require deep or highly overparameterized networks.

Overall assessment. Taken together, these results demonstrate that the core behaviour of the SABR implied-volatility surface, including its dependence on maturity, correlation, vol-of-vol, and the elasticity parameter β, can be replicated to high fidelity using the Hagan analytical expansion alone. The trained networks are stable, smooth, and accurate across the full parameter range. This confirms that the first phase of the original two-stage pipeline of [McGhee, 2020] is reproducible and reliable without resorting to finite-difference solvers, and that the analytical formula provides a sufficiently rich and well-behaved training signal for data-driven surrogates on compact domains.

5.2 Conditional integration method

The empirical behaviour of the neural network trained on integration-based implied volatilities differs significantly from the results obtained using the Hagan approximation or the 2FDM solver. Although the conditional-integration method of [McGhee, 2011] is appealing because of its simplicity and low computational cost, the numerical structure of the resulting option prices poses challenges for the fine-tuning stage.

Irregularities in the implied-volatility surface. The integration method computes each option price using a separate one-dimensional quadrature. While this produces accurate pointwise values, the resulting implied-volatility surface lacks the global smoothness of the Hagan and, in particular, the 2FDM datasets. As a consequence, the ANN can learn the overall shape of the smile but the fine-tuning step is unable to eliminate the small oscillations introduced by the integration routine, as visible in Figures 14–16.

Sensitivity to challenging parameter regimes. The shortcomings become more pronounced in regions where the SABR transition density is skewed or heavy-tailed, such as short maturities combined with large volatility-of-volatility. In these regimes, numerical quadrature becomes more delicate, and the implied volatilities produced by the integration method exhibit higher local variance. Since the fine-tuning network is trained to refine a smooth baseline (the Hagan surface), it struggles to learn corrections that themselves are noisy or inconsistent across parameters.

Impact on Phase 2 refinement. The refinement stage introduced by [McGhee, 2020] implicitly relies on the assumption that the target surface is both coherent and smooth. This is true for the 2FDM solver, whose PDE structure enforces global regularity, but not for the integration-based dataset. Consequently, simply increasing the network width does not deliver the uniform error reduction observed in the 2FDM experiments. The ANN is ultimately limited by the variability present in the target it is asked to learn.

Overall assessment. The integration method provides a fast and flexible way to generate implied volatilities, but its lack of global coherence makes it less suitable for the fine-tuning

strategy. The neural network learns the overall geometry of the surface but cannot compensate for the local inconsistencies introduced by the numerical quadrature.

5.3 Finite-difference scheme method

The empirical performance of the neural network trained on 2FDM-based implied volatilities aligns closely with the behaviour reported in [McGhee, 2020]. The finite-difference solver produces a globally smooth and internally consistent volatility surface, which greatly facilitates the fine-tuning stage. Compared with the integration-based dataset, the 2FDM labels exhibit substantially lower numerical variance and form a coherent target for the ANN refinement.

Note. The FD surfaces shown in Section 4 were generated with the initial version of the solver (see explanation in Section 2.4). A minor grid-alignment issue was identified and corrected after producing the figures; preliminary tests with the updated solver confirm that the new FD surfaces are even smoother, and the ANN refinement improves accordingly. The qualitative conclusions reported here remain unchanged.

Smoothness and internal coherence. A key advantage of the 2FDM solver is that all strikes at a given maturity are generated simultaneously by solving a single two-dimensional PDE. This enforces global structural consistency across the smile and across parameters: nearby strikes vary smoothly, correlation effects in the (F, σ) directions are handled naturally, and numerical noise remains low. As a result, the ANN encounters a stable and smooth target surface, allowing the refinement stage to make systematic improvements over the Hagan baseline.

Robustness across SABR parameters. The FD-based implied volatilities remain stable even in challenging regions of the SABR parameter space, such as short maturities, large volatility of volatility, and strong correlation. Because the PDE solution captures the full joint dynamics of the forward and volatility, it handles skewed or heavy-tailed transition densities without introducing the strike-by-strike inconsistencies seen in the integration method. This robustness is reflected in the error surfaces, where the ANN achieves uniform improvements across strikes and maturities.

Effective Phase 2 refinement. The fine-tuning strategy of [McGhee, 2020] relies on having a high-quality target surface that corrects the systematic biases of the Hagan approximation while preserving global smoothness. The 2FDM dataset satisfies these requirements: the refinement network learns a consistent correction term that transfers smoothly across the input domain. Unlike in the integration-based experiments, increasing the network width yields a clear and monotonic reduction in error, confirming that the ANN is able to exploit the structure present in the FD labels.

Overall assessment. The finite-difference solver provides the most reliable high-fidelity target for training neural SABR surrogates. Its ability to generate globally smooth, PDE-consistent implied volatilities makes it well suited for the refinement stage and allows the ANN to reach sub-basis-point accuracy over large regions of the parameter space. Although computationally more expensive than the integration method, the quality of the resulting surface leads to substantially better neural-network performance.

6 Conclusion

In this work, we reproduced and extended the neural-network approximation framework presented in [McGhee, 2020] for the SABR stochastic-volatility model. We systematically evalu-

ated its behaviour across elasticity parameters, hidden-layer widths, and alternative high-fidelity training targets.

Our experiments confirm that the Hagan-based first training stage is remarkably robust. Whether β is treated as fixed ($\beta = 1$) or included as an input feature ($\beta \in [0, 1]$), the neural surrogate accurately reproduces the implied-volatility surface across the full elasticity range, including the normal and intermediate regimes. This demonstrates that the ANN architecture is expressive enough to interpolate smoothly between qualitatively distinct volatility shapes.

The width-ablation study further shows that relatively small networks already capture the global structure of the volatility smile, while networks with insufficient capacity exhibit systematic underfitting: they reproduce the qualitative shape but fail to resolve curvature and wing behaviour. Beyond a moderate number of hidden units, additional capacity yields diminishing returns. In particular, networks with widths above roughly $W \approx 500$ achieve near-plateau performance, and further increases (up to $W = 3000$) provide no measurable improvement. This indicates where the model reaches an effective balance between accuracy and computational cost.

The two-factor finite-difference (2FDM) refinement behaves consistently with the findings of [McGhee, 2020]: the fine-tuning stage successfully transfers the smoothness of the PDE solution to the neural network, producing highly accurate and coherent implied-volatility surfaces. A numerical bias introduced by the initial ADI time-stepping scheme was identified in the first version of the FD solver. Preliminary runs with the corrected Craig–Sneyd implementation yield smoother and more accurate reference surfaces, further reinforcing the qualitative conclusions reported here.

In contrast, fine-tuning on the conditional-integration method does not yield the same benefits. Although the integration engine produces pointwise accurate prices, its lack of global smoothness propagates into the implied-volatility labels, preventing the network from learning a stable correction. The ANN captures the broad smile shape but cannot reliably eliminate the local oscillations inherited from the integration routine.

Overall, our study confirms that the original two-phase method—Hagan approximation followed by 2FDM refinement—remains the most effective approach. At the same time, the first training stage alone already provides a robust and computationally efficient surrogate for a wide range of SABR specifications. The extension to $\beta \in [0, 1]$ further demonstrates that neural surrogates generalise smoothly across elasticity regimes, while the width-ablation study clarifies the minimal architectural complexity required for accurate approximation.

These results offer practical guidance for selecting network sizes, training pipelines, and surrogate models when deploying neural approximations of the SABR model in calibration or risk-management applications.

References

[Hagan et al., 2002] Hagan, P. S., Kumar, D., Lesniewski, A. S., and Woodward, D. E. (2002). Managing smile risk. *Wilmott Magazine*, pages 84–108.

[Lyu, 2020] Lyu, F. (2020). Moments of integrated lognormal diffusions and applications. *SSRN Electronic Journal*.

[McGhee, 2011] McGhee, W. A. (2011). An efficient implementation of stochastic volatility by the method of conditional integration. In *ICBI Conference*, Paris.

[McGhee, 2020] McGhee, W. A. (2020). An artificial neural network representation of the sabr stochastic volatility model. *Journal of Computational Finance*, 25(2).